Reglas de dependencia

De una aplicación n-capas a componentes desacoplados

Caso de estudio: reserva de citas con Node.js, Express y PostgreSQL

SEMANA 7
Cliente-servidor

SEMANA 8
Arquitectura en capas

SEMANA 9
Reglas de dependencia

Separar los archivos no elimina todo el acoplamiento

La aplicación ya tiene cuatro capas, pero el caso de uso conoce repositorios concretos.

El avance

Las rutas HTTP, las reglas de negocio y las consultas SQL están distribuidas en módulos distintos.

La dependencia pendiente

src/aplicacion/citaService.js

```
const reglas = require('../dominio/reglasDeAgenda');

const citaRepository =
require('../persistencia/citaRepository');

const profesionalRepository =
require('../persistencia/profesionalRepository');
```

Esas importaciones obligan al servicio a conocer el detalle de persistencia.

Fuentes: repositorio demo-cliente-servidor, citaService.js; Cervantes Maceda et al. (2016, §3.3).

Dependencia y regla de dependencia

UML permite describir una relación; el diseño arquitectónico establece cuáles relaciones admitimos.

DEPENDENCIA EN UML

Relación cliente–proveedor

Un elemento cliente requiere otro elemento para su especificación o implementación. La flecha de dependencia señala al proveedor.

Se representa con una línea discontinua y punta abierta.

REGLA DE DEPENDENCIA

Restricción del diseño

Expresa qué módulos pueden conocer a otros y qué relaciones deben evitarse. Debe formularse de modo que pueda verificarse en el código.

Ejemplo: aplicación no debe importar persistencia concreta.

Una dependencia puede existir sin ser incorrecta: depende de la frontera arquitectónica que atraviesa.

Fuentes: OMG (2017), UML 2.5.1, §7.7; Martin (2012), The Clean Architecture.

Cómo identificar y escribir una regla

El análisis empieza con las importaciones reales y termina con una restricción que puede comprobarse.

Archivo examinado: src/aplicacion/citaService.js

```
const reglas = require('../dominio/reglasDeAgenda');
```

Permitida

```
const citaRepository = require('../persistencia/citaRepository');
```

Acoplamiento

1 Identificar el cliente

citaService.js

2 Identificar el proveedor

citaRepository.js

3 Evaluar la regla

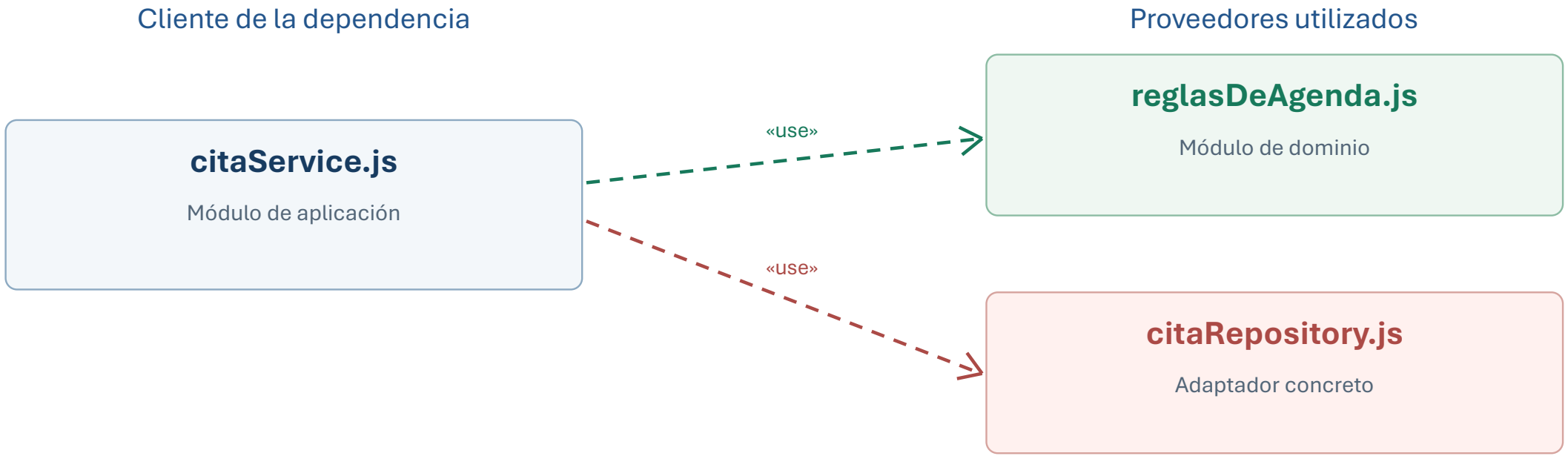
¿Detalle o contrato?

Regla verificable: La aplicación no importa implementaciones de persistencia; recibe repositorios mediante DI.

Fuentes: código de src/aplicacion/citaService.js; Martin (2012); Fowler (2004).

Cómo leer una dependencia UML

La flecha parte del elemento que depende y apunta hacia el elemento utilizado.



Línea discontinua

Punta abierta

«use» identifica dependencia de uso

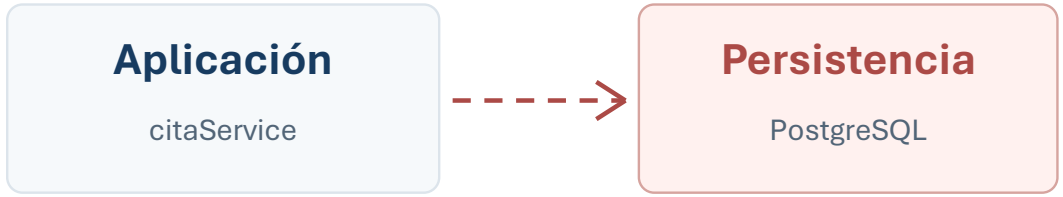
Fuente: Object Management Group (2017), UML 2.5.1, §§7.7.1 y 7.7.4. Ejemplo: código del curso.

DIP: el caso de uso no debe conocer PostgreSQL

La aplicación define lo que necesita; el adaptador concreto suministra esas operaciones.

Dependencia actual

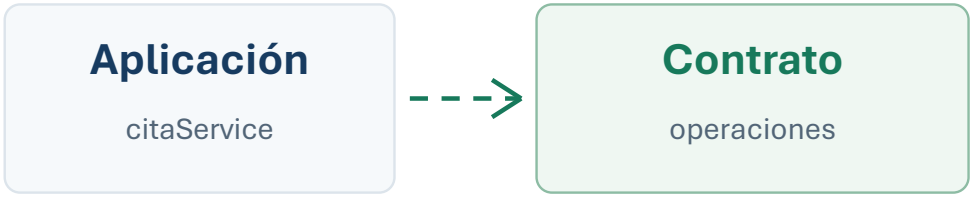
Acoplamiento con el repositorio concreto



Cambiar la implementación obliga hoy a editar el servicio.

Diseño propuesto

El servicio trabaja con operaciones esperadas



El repositorio PostgreSQL o el simulado satisface el mismo contrato estructural.

Fuentes: Martin (2012), The Clean Architecture; Cervantes Maceda et al. (2016, pp. 34–35).

DI: las dependencias se entregan desde afuera

La factoría del servicio recibe repositorios; server.js decide qué implementaciones utilizar.

Dentro de aplicación

```
function crearCitaService({
  citaRepository,
  profesionalRepository
}) { /* reglas y casos de uso */ }
```

En el punto de composición

```
const servicio = crearCitaService({
  citaRepository,
  profesionalRepository
});
```

DIP establece qué dependencia conviene evitar. DI permite entregar las implementaciones sin que el servicio las construya.

Fuente: Fowler (2004), Inversion of Control Containers and the Dependency Injection Pattern.

El cambio en el repositorio es acotado

No se modifica el contrato HTTP ni las reglas del dominio.

01	NUEVO	`src/aplicacion/contratos/repositorios.js`	Explicita los métodos requeridos por el caso de uso.
02	CAMBIAR	`src/aplicacion/citaService.js`	Recibe los repositorios y conserva las reglas.
03	CAMBIAR	`src/presentacion/citasRoutes.js`	Recibe el servicio como parámetro de una factoría.
04	CAMBIAR	`server.js`	Construye las piezas y las conecta al iniciar Express.

Permanecen sin cambios: reglasDeAgenda.js, los repositorios PostgreSQL y public/index.html.

Fuente: código del proyecto de referencia y guía de laboratorio de la semana 9, apartado 5.

Cómo comprobar que el desacoplamiento es real

La evidencia debe observarse en el código y en las pruebas, no solo en el diagrama.

1

Reemplazo del adaptador

El servicio acepta repositorios en memoria sin modificar la lógica de aplicación.

2

Pruebas aisladas

Tres pruebas guiadas y una cuarta diseñada por cada equipo, sin conectar Supabase.

3

API sin regresiones

Las rutas y respuestas HTTP conservan su comportamiento con PostgreSQL.

Entrega: cambios en GitHub, resultados de pruebas y diagramas UML del repositorio del equipo.

Fuente: guía del laboratorio de la semana 9, apartado 8; Fowler (2004); Martin (2012).

Referencias bibliográficas

- Cervantes Maceda, H., Velasco-Elizondo, P. y Castro Careaga, L. (2016).

  Arquitectura de software: conceptos y ciclo de desarrollo. Cengage Learning. Cap. 3, §3.3.

- Martin, R. C. (2012). The Clean Architecture.

  The Clean Code Blog. blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

- Fowler, M. (2004). Inversion of Control Containers and the Dependency Injection Pattern.

  martinfowler.com/articles/injection.html

- Object Management Group. (2017). Unified Modeling Language, Version 2.5.1.

  Secciones 7.7.1 y 7.7.4. omg.org/spec/UML/2.5.1/PDF

- Proyecto de referencia del curso. (2026). Demo cliente-servidor.

  Aplicación de reserva de citas. github.com/ebuitrago/demo-cliente-servidor

Citas abreviadas junto al contenido. Enlaces activos para consultar las fuentes en línea.